Neural Networks and Deep Learning (2026)
Assignment (3):

Shiraz University

Due date: 10th of Tir 23:59 [1/June (2026)]

1 Conditional GAN on Fashion-MNIST

1.1 Introduction

Generative Adversarial Networks are powerful models for learning how to generate realistic data. A standard GAN learns to generate images from random noise, but it does not allow direct control over the type of image being generated. A Conditional GAN, or cGAN, solves this limitation by adding extra information, such as a class label, to both the generator and the discriminator. This allows the model to generate images belonging to a specific class, for example a sneaker, bag, trouser, or shirt in the Fashion-MNIST dataset.

1.2 Problem Statement

In this homework, you will implement a Conditional GAN using the Fashion-MNIST dataset. The goal is to train a generator that can create Fashion-MNIST-like images conditioned on class labels. The generator should receive random noise together with a class label and generate an image corresponding to that label. The discriminator should receive an image and a class label, then decide whether the image is real or generated.

You only need to use the training set of Fashion-MNIST for this homework. The test set is optional and is not required, because the main goal is image generation rather than classification accuracy.

Dataset download link:

https://github.com/zalandoresearch/fashion-mnist

or on Intranet link:

[DOWNLOAD DATASETS]

You may use a standard dataset loader or manually read the raw Fashion-MNIST files. If you use only part of the dataset to reduce training time, you must clearly report the percentage or number of samples used.

1.3 Tasks

1. Implement a Conditional GAN for Fashion-MNIST. Your model must include a generator, a discriminator, and a complete training loop.
2. The generator must use both random noise and class-label information to generate class-conditioned images.
3. The discriminator must use both the image and class-label information to judge whether the image is real or fake.
4. Train the model for a reasonable number of epochs. You may choose the number of epochs based on your computational resources, but you must report your settings clearly.
5. Generate image samples from different Fashion-MNIST classes and show them in a grid. The generated samples should include multiple classes, not only one class.
6. Plot the generator loss and discriminator loss during training.

7. Discuss the behavior of the losses. Explain whether the losses were stable, unstable, fluctuating, or showed signs of imbalance between the generator and discriminator.
8. Compare the generated images at different stages of training, if possible, for example early training versus final training.
9. Discuss which classes were generated better and which classes were harder to generate.
10. Suggest possible improvements to obtain better generated images.

1.4 Report Requirements

Your final submission must include:

- The source code or notebook.
- A short explanation of your cGAN architecture.
- A generated image grid showing samples from different Fashion-MNIST classes.
- A plot of generator loss and discriminator loss.
- A discussion of the loss behavior.

A short discussion answering the following questions:

- What is the main difference between a GAN and a cGAN?
- How did you provide class information to the generator and discriminator?
- Which classes were easier or harder to generate?
- How did the generated images change during training?
- How did the generator and discriminator losses behave?
- How could the results be improved?

2 Grayscale Image Colorization using Convolutional Autoencoders

2.1 Introduction

Image colorization is the process of automatically predicting plausible color information for grayscale images. This is a challenging problem in computer vision because a single grayscale image may correspond to several possible valid color images. For example, the same grayscale shirt could plausibly be red, blue, green, or black.In this homework, you will implement a Convolutional Autoencoder for automatic image colorization. The model receives a single-channel grayscale image as input and reconstructs a three-channel RGB image as output. The encoder learns hierarchical visual features from the grayscale image, while the decoder uses these features to predict the missing color information.

Training is performed in a self-supervised reconstruction setting. The grayscale input images are created from the original RGB images, and the original RGB images are used as target outputs. This task reinforces important deep learning concepts such as encoder-decoder architectures, convolutional feature extraction, image reconstruction losses, and evaluation of generative image tasks.

2.2 Problem Statement

In this homework, you are provided with a dataset of 500 natural images captured from everyday scenes, including landscapes, people, animals, objects, and other real-world images.

The dataset is divided as follows: Training set: 490 images, Test set: 10 images

Dataset download link:

[DOWNLOAD DATASETS]

Your goal is to design and train a Convolutional Autoencoder that learns a mapping from grayscale images to their corresponding full-color RGB versions. The model should learn to predict plausible color information while preserving the structure and content of the input image.

2.3 Tasks

1. Load the image dataset and preprocess all images. Resize the images to a fixed resolution and normalize pixel values to the range [0, 1].
2. Convert the original RGB images to grayscale. These grayscale images will be used as the model inputs, while the original RGB images will be used as the target outputs.
3. Design and implement a Convolutional Autoencoder. The encoder should use convolutional layers to process the 1-channel grayscale input and extract hierarchical visual features. The decoder should reconstruct a 3-channel RGB image.
4. Train the model using grayscale images as inputs and original RGB images as targets. You may use a reconstruction loss such as Mean Squared Error loss or L1 loss. Train the model for a reasonable number of epochs and monitor the training and validation loss.
5. Evaluate the trained model on the 10 test images. For each test image, generate the colorized output from its grayscale version.
6. Visualize the results for the test images. Each result should show the following images side by side:
    - Grayscale input image
    - Colorized output produced by the model
    - Original RGB image
7. Compute and report the following quantitative evaluation metrics on the test set:
    - Mean Squared Error
    - Peak Signal-to-Noise Ratio
    - Structural Similarity Index
8. Plot the training and validation loss curves.
9. Discuss the quality of the generated colorized images. Explain which images were colorized better and which images were more difficult.
10. Discuss the limitations of the model. Your discussion should include possible reasons for inaccurate or blurry colorization results and suggest possible improvements.

2.4 Report Requirements

Your final submission must include:

- The source code or notebook.
- A short explanation of your Convolutional Autoencoder architecture.
- A description of the preprocessing steps, including resizing, normalization, and grayscale conversion.
- A plot of training and validation loss.
- A visualization of the 10 test results showing:
- grayscale input,model colorized output,original RGB image.
- A table or clear report of the test-set evaluation metrics:
- MSE, PSNR, SSIM.

A short discussion answering the following questions:

- How does the encoder-decoder structure help in image colorization?

- What loss function did you use and why?
- Which test images were colorized better?
- Which test images were more difficult to colorize?
- What are the main limitations of using a simple Convolutional Autoencoder for image colorization?
- How could the colorization results be improved?

2.5 Bonus Task: Adversarial Autoencoder for Latent Space Learning

As an optional bonus task, you may implement an Adversarial Autoencoder on the RGB images. In this part, the model should reconstruct RGB images from RGB inputs and use an adversarial discriminator in the latent space to force the encoded latent vectors to match a chosen prior distribution, such as a Gaussian distribution or any other distribution you desire.

For the bonus task, you may:

- Train an RGB Adversarial Autoencoder.
- Plot the learned latent distribution using PCA or histograms.
- Compare the encoded latent distribution with the selected prior distribution.
- Generate random samples from the prior distribution.
- Perform latent-space interpolation between two test images.

✓ **<u>This bonus task is optional and will be graded separately from the main colorization task.</u>**

Notes:

- Allowed programming languages: Python, MATLAB
- Any sign of cheating would result in a zero grade for this assignment.
- You should upload your submissions at:
  https://quera.org/course/add_to_course/course/27529/
  Password: nn2026
  All of the files should be in a ZIP file named in this format:
  "FirstNameFamilyName-SudentNumber.zip"
  Ex: "AmirZamani-4053040.zip"
- Your reports should be in a PDF file including: key points of your implementation, explanation of your chosen approach, reports of your final results and answers of assignment questions (if given).
- If you do not have **Anaconda** or **Python** installed, you can download Anaconda from the following link: [**DOWNLOAD LINK**]
- Use the following **domestic PyPi mirror** to install the required libraries:
  https://package-mirror.liara.ir/repository/pypi/simple
  For more information about this mirror, visit: **https://liara.ir/mirrors/pypi/**
  Example: "`pip install minisom --index-url https://package-mirror.liara.ir/repository/pypi/simple`**"**
  If this mirror doesn't work, try Runflare instead: **https://runflare.com/mirrors/pypi-mirror/**
